ИННОВАЦИИ В ФИНТЕХЕ:
ТЕОРИЯ И ПРАКТИКА

Безопасность цепочки поставки

Станислав
Сумин



НСПК | МИР | СБП

Повестка

Разберём путь атаки, ключевые кейсы, уровни защиты, реагирование на инцидент и практический чек-лист разработчика



01 Общая информация

02 Типы атак

03 Кейсы

04 Методы защиты

05 Инцидент-ответ

06 Чек-лист разработчика

раздел

01

Общая информация



Почему это важно именно вам?

Ваш код — это только часть продукта

Современное приложение собирается из множества компонентов, которым вы доверяете доступ к сборке, секретам и рабочей среде

01 Состав кода

80–90% современного проекта в среднем составляют транзитивные зависимости, а не ваша бизнес-логика.

Вы пишете бизнес-логику, но запускаете сотни чужих компонентов

02 Атаки растут

Публичные registry и автоматические сборки превращают одну скомпрометированную зависимость в массовый канал атаки.

Чем быстрее поставка, тем меньше времени на ручную проверку

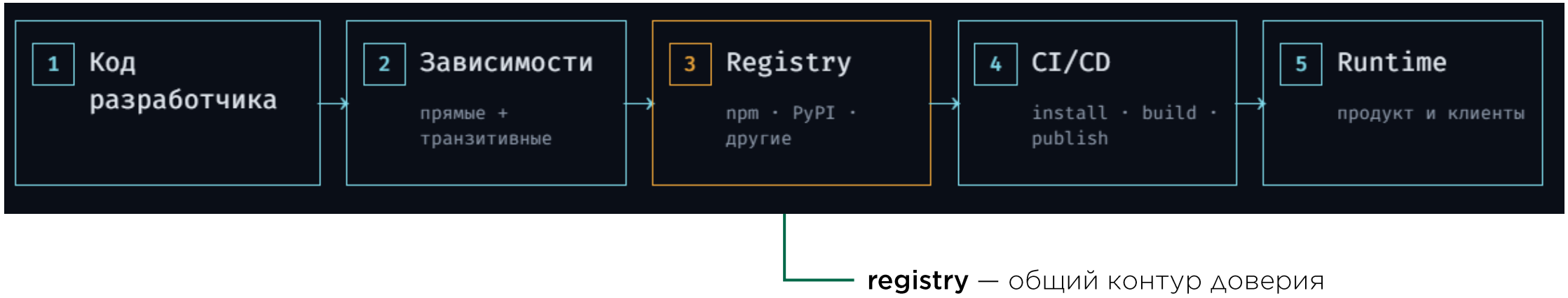
03 Цена ошибки

Последствия выходят далеко за пределы одного пакета: утечки секретов, бэкдоры, простой и блокировки

Главный вывод:

зависимость — это не просто строка в манифесте, а часть доверенной цепочки поставки

Карта цепочки поставки



! Уязвимость может попасть на любом из пяти шагов — от строки кода до прод-окружения

Масштаб угрозы в цифрах

Заявленный масштаб*

2025-2026

454 тыс. +

заявленных вредоносных пакетов

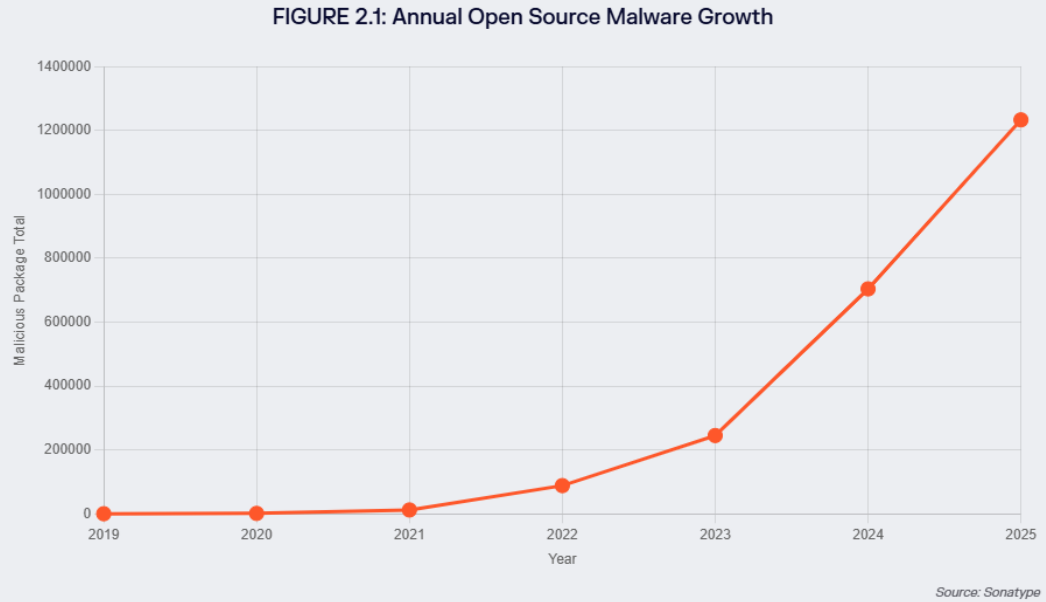
≈90%

доля npm среди заявленных пакетов

73–75%

заявленный рост год к году

* — https://www.sonatype.com/state-of-the-software-supply-chain/2026/open-source-malware



Вывод: масштаб npm превращает единичную компрометацию пакета в потенциально массовый инцидент

Почему именно npm?

npm = огромный масштаб + автоматическое доверие

- Миллионы пакетов и проектов
- Глубокий граф транзитивных зависимостей
- Быстрая публикация и обновление
- Популярные пакеты устанавливаются миллионами раз
- Один скомпрометированный пакет → множество проектов



> Экосистема оптимизирована под скорость публикации и повторного использования. Это же создаёт удобный канал для массового распространения

Почему именно npm install?

npm install — это не просто download

01 Разрешить граф зависимостей

02 Скачать пакеты

03 Выполнить install lifecycle scripts

04 Запустить сборку нативных модулей

05 Сформировать node_modules



Если зависимость скомпрометирована, она может атаковать ещё до запуска приложения

раздел

02

Типы атак

Пять способов, которыми злоумышленники компрометируют цепочку поставки npm



Типы атак на цепочку поставки

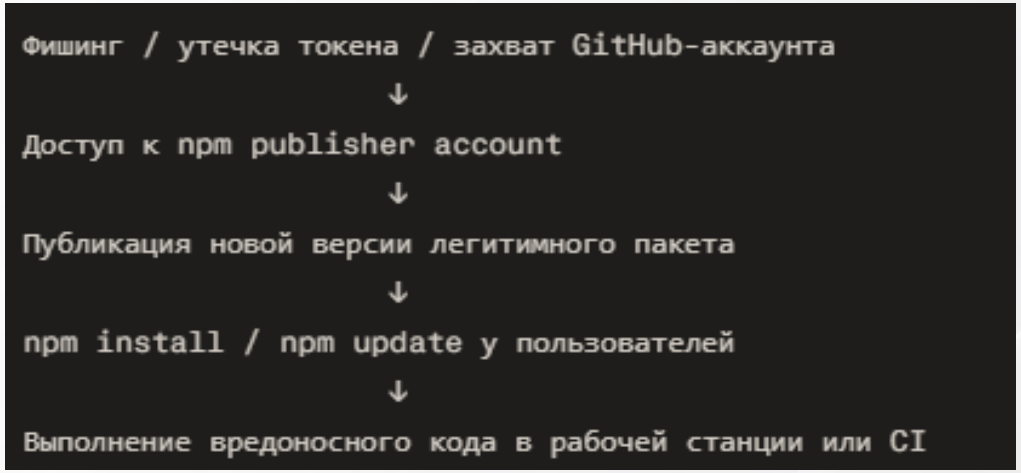
Тип	Механика	Пример
01 Угон аккаунта	Кража credentials мейнтейнера для публикации вредоносных версий легитимного пакета	Выпущена вредоносная версия легитимного пакета
02 Тайпсквоттинг	Публикация пакета с именем, похожим на популярный, в расчёте на опечатку при установке	Разработчик установил подделку из-за опечатки
03 Dependency confusion	Публичный пакет с именем внутреннего приватного пакета — резолвер подтягивает чужой код	CI скачал чужой пакет вместо приватного
04 Malicious updates	Вредоносный код добавляется в новую версию уже доверенного пакета	Автообновление принесло бэкдор

Угон аккаунта

Что происходит

Угон аккаунта — это получение злоумышленником доступа к учётной записи мейнтейнера или организации, которая имеет права публиковать пакет в npm

Типовая цепочка атаки



После захвата аккаунта атакующий может

- выпустить новую версию уже доверенного пакета
- изменить package.json и добавить lifecycle-скрипт
- добавить зависимость на вредоносный пакет
- изменить владельцев пакета, чтобы сохранить доступ
- опубликовать несколько вредоносных версий за короткий период
- получить доступ к токенам npm, GitHub, CI/CD и registry

Тайпсквоттинг

Что происходит

Тайпсквоттинг — публикация пакета с названием, визуально или фонетически похожим на популярную библиотеку

Цель — поймать ошибку человека

Типовая цепочка атаки

```
Популярное имя пакета
        ↓
Опечатка / неверный tutorial / AI-ответ / copy-paste
        ↓
npm install <похожее имя>
        ↓
Устанавливается пакет атакующего
        ↓
Lifecycle-скрипт или вредоносный код
```

Примеры названий

Ожидаемый пакет	Поддельный вариант	Что изменено
lodash	l0dash	Буква o заменена цифрой 0
axios	axioss	Добавлена лишняя буква
express	expres	Пропущена буква

Dependency confusion

Что происходит

Dependency confusion — это ошибка в маршрутизации или разрешении зависимостей между приватным и публичным registry

- **Компания** может иметь внутренний пакет, например:text@company/auth-utils

- **Разработчики** ожидают, что он придёт из внутреннего Nexus, Artifactory, GitHub Packages или другого private registry. Но если настройка .npmrc, registry proxy или приоритет источников неверна, npm может скачать пакет из публичного registry

- **Атакующий** публикует в публичном npm пакет с тем же названием или с именем, похожим на внутреннее, зачастую с более высокой версией. Если build-система разрешит именно публичный источник — в проект попадёт код атакующего



раздел

03

Кейсы

Четыре инцидента 2025–2026: черви, угнанные токены, бэкдоры и компрометация мейнтейнеров



Shai-Hulud: самокопирующийся червь



24 ноября 2025 года

вторая волна

Масштаб

796 скомпрометированных пакетов; более 20 млн загрузок в неделю

Механизм

Зловред читает собственный код и встраивает себя в другие пакеты через похищенные npm-токены — без связи с C2-сервером

Урон

Данные более 500 GitHub-пользователей и 150+ организаций похищены и опубликованы в публичных репозиториях

Ключевой риск

self-replication убирает зависимость от центра управления

CanisterWorm: бэкдор с блокчейн-управлением



20 марта 2026 года

атрибуция: TeamPCP

Масштаб

Свыше 60 пакетов и 130+ артефактов на старте, включая пространства имён крупных компаний

Распространение

Похищенные npm-токены автопубликуют вредоносные patch-версии во всех пакетах жертвы

Особенность

Command-and-control построен на децентрализованном блокчейне ICP: обычное отключение C2 не работает

Ключевой риск

инфраструктура управления живёт вне привычного домена блокировки

jscrambler: компрометация через угнанный токен



11 июля 2026 года

Причина

Скомпрометированное устройство разработчика раскрыло SSH-ключ и npm-токен публикации

Обнаружение

Вредоносная версия найдена через 6 минут после публикации благодаря автоматическому мониторингу

Последствия

Preinstall-хук ставил скрытые бинарники под Linux, macOS и Windows; почти 1500 загрузок за два часа

keyv / cacheable: угон аккаунта разработчика



4 августа 2026 года

компрометация GitHub-аккаунта основного мейнтейнера

Масштаб

keyv — 127 млн загрузок в неделю, семейство пакетов (keyv, cacheable, flat-cache, file-entry-cache и другие) — свыше 500 млн загрузок в неделю совокупно

Распространение

Червь расползся на 444+ пакета и 1300+ версий, включая пакеты сторонних компаний (Deliveroo, Qlik, Picsart)

Опасная деталь

Вредонос имел "dead man's switch" — реагировал на попытку отзыва токена, что делало стандартный playbook ротации credentials опасным

раздел

04

Методы защиты

Три уровня контроля — от базовой гигиены до организационных барьеров



Уровень 1: базовая гигиена

Мера	Описание
01 Lockfile в репозитории	package-lock.json / yarn.lock всегда закоммичены и проверяются
02 Пин версий	Без широких диапазонов (^, ~) для критичных зависимостей
03 Осознанный update	Регулярное обновление с обязательной проверкой changelog

Уровень 2: верификация происхождения

Мера	Описание
01 Sigstore / cosign	Криптографическая подпись артефактов сборки
02 npm provenance	Привязка опубликованного пакета к коммиту и CI-запуску
03 SLSA	Уровни зрелости цепочки поставки как ориентир для оценки риска

Уровень 3: организационный контроль

Мера	Описание
01 Приватный прокси-registry	Verdaccio, Artifactory или Nexus как единая точка входа пакетов
02 Политики block/allow	Явный список разрешённых пакетов, блокировка остального
03 Сканеры в PR	Проверка новых зависимостей до мержа в основную ветку

раздел 05

Инцидент-ответ



НСПК | МИР | СБП

Инцидент-ответ: если пакет уже установлен

01 Изолировать машину от сети

02 Не отзывать токены сразу — некоторые вредоносы реагируют деструктивно на revoke, сначала найти и удалить механизмы персистентно

03 Собрать артефакты для форензики

04 Провести ротацию credentials только с чистой машины

06 Уведомить команду и registry

раздел 06

Чек-лист разработчика



НСПК | МИР | СБП

Почему это важно именно вам?

- Lockfile закоммичен и проверяется в CI
- Критичные зависимости закреплены версиями
- Новые зависимости сканируются перед мержем
- CI/CD токены с минимальными правами и ограниченным сроком жизни
- Есть план действий на случай инцидента

ИННОВАЦИИ В ФИНТЕХЕ:
ТЕОРИЯ И ПРАКТИКА

Спасибо
за внимание!

Станислав
Сумин

НСПК | МИР | СБП

